



System.out.println

Módulo: Proyecto

Curso: 3º de DAW

David Delgado León

Propuesta proyecto 25/26

■
;

1. Título del proyecto.....	2
2. Resumen del proyecto.....	2
3. Descripción del Problema / Necesidad.....	2
4. Objetivos.....	3
4.1 General.....	3
4.2 Específicos.....	3
5. Alcance del Proyecto.....	3
6. Usuarios del Sistema.....	4
7. Tecnologías a Utilizar.....	4
8. Requisitos del Sistema.....	5
8.1 Requisitos Funcionales (RF).....	5
8.2 Requisitos No Funcionales (RNF).....	5
9. Áreas del ciclo formativo desarrolladas.....	6

1. Título del proyecto

Nombre: Eldera

Subtítulo: Sistema de Gestión Integral para Residencias de Mayores.

Identificación del proyecto:

- Nombre y Apellidos Participante: David Delgado León
- Ciclo formativo: Desarrollo de Aplicaciones Web (DAW)
- Centro educativo: IES Albarregas, Mérida.
- Fecha: Diciembre 2025

2. Resumen del proyecto

Eldera es una aplicación web moderna diseñada para digitalizar y optimizar la gestión diaria de una residencia de ancianos. El sistema permite abandonar el uso de papel y hojas de cálculo dispersas, centralizando la información de los residentes en una base de datos segura y accesible. Está dirigido al personal administrativo y sanitario de residencias, esperando como resultado una mejora en la eficiencia operativa, reducción de errores en la medicación y datos de contacto, y un acceso rápido a la información crítica en situaciones de emergencia.

3. Descripción del Problema / Necesidad

Contexto: Actualmente, muchas residencias pequeñas y medianas gestionan la información de sus residentes (datos personales, historial médico, contactos de emergencia) mediante fichas de papel o archivos Excel desconectados.

Situación actual: Esto provoca duplicidad de datos, dificultad para buscar información rápidamente, riesgos de pérdida de documentos y falta de control sobre quién accede a los datos sensibles.

Justificación: Desarrollar Eldera vale la pena porque democratiza el acceso a software de gestión profesional. Al usar tecnologías web modernas y contenedores, se ofrece una solución robusta, segura y fácil de desplegar que moderniza instantáneamente la infraestructura de cuidados de la residencia.

4. Objetivos

4.1 General

Desarrollar e implementar una aplicación web full-stack escalable para la gestión integral de la información de residentes en centros de mayores, garantizando la seguridad de los datos y la accesibilidad desde múltiples dispositivos.

4.2 Específicos

1. Desarrollar una API RESTful robusta con Python y FastAPI para la lógica de negocio.
2. Implementar una interfaz de usuario moderna y responsiva utilizando React y Tailwind CSS.
3. Integrar un sistema de autenticación y autorización basado en roles (RBAC) para diferenciar permisos entre el personal.
4. Optimizar el despliegue y entorno de desarrollo mediante la contenedorización con Docker y Docker Compose.
5. Facilitar la migración de datos mediante funcionalidades de importación y exportación masiva.

5. Alcance del Proyecto

Incluirá:

- Módulo de Autenticación y Seguridad (Login, JWT).
- Módulo de Gestión de Residentes (Altas, Bajas, Modificaciones, Listados).
- Módulo de Búsqueda Avanzada y Filtrado.
- Módulo de Importación/Exportación de datos.
- Interfaz adaptada a dispositivos móviles (tablets de enfermería).

Algunos apartados que no incluirá (en esta fase MVP):

- Gestión de facturación y cobros a familiares.
- Control de farmacia y medicación(inventario detallado).

- Seguimientos de los diferentes especialistas

Tiempo estimado: 4-5 meses.

Límites: El sistema requiere conexión a red local o internet para funcionar (no offline-first en v1).

6. Usuarios del Sistema

- Administrador / Director: Tiene acceso total. Puede registrar nuevos usuarios (empleados), dar de alta/baja residentes, y exportar informes completos. Necesita visión global y herramientas de gestión de datos.
- Enfermero: Acceso un poco más limitado que el de administrador, puesto que no puede eliminar residentes pero puede modificar datos de los residentes.
- TCAE: Acceso limitado. Su función principal es consultar fichas de residentes, ver números de habitación y contactos de emergencia. Necesitan una interfaz rápida y fácil de usar en tablets/móviles. En futuras actualizaciones podrán registrar información respecto a las funciones que les competen.

7. Tecnologías a Utilizar

Frontend (Cliente):

- Lenguaje: JavaScript (ES6+).
- Framework: React 19 (con Vite para build tool).
- Estilos: Tailwind CSS v4 (Diseño utility-first).
- Librerías: Axios (HTTP), React Router (Navegación), React Icons.

Backend (Servidor):

- Lenguaje: Python 3.11.
- Framework: FastAPI (Alto rendimiento, validación automática).
- ORM: SQLAlchemy (Abstracción de base de datos).
- Validación: Pydantic.

Base de Datos:

- Motor: PostgreSQL 15 (Relacional, robusta).

Herramientas y DevOps:

- Control de Versiones: Git & GitHub.
- Contenedorización: Docker y Docker Compose (para orquestar Frontend, Backend y BD).
- Pruebas API: Postman / Scripting Python.
- IDE: Visual Studio Code.

8. Requisitos del Sistema

8.1 Requisitos Funcionales (RF)

- RF1 - Gestión de Sesiones: El sistema permitirá a los usuarios identificarse mediante usuario y contraseña, recibiendo un token de acceso seguro (JWT).
- RF2 - Gestión de Residentes: El usuario con permisos podrá Crear, Leer, Actualizar y Eliminar (CRUD) fichas de residentes.
- RF3 - Búsqueda: El sistema permitirá filtrar residentes por nombre, apellido o número de habitación en tiempo real.
- RF4 - Importación Masiva: El administrador podrá cargar múltiples residentes desde un archivo estándar.
- RF5 - Exportación de Datos: El sistema permitirá descargar el listado actual de residentes en formato CSV o JSON.
- RF6 - Control de Acceso: El sistema restringirá ciertas acciones (como borrar residentes) únicamente a usuarios con rol de 'admin'.
- RF7 - Generación de listados automáticos: A partir de la ficha de cada residente se podrán crear listados dinámicos que estén actualizados continuamente.

8.2 Requisitos No Funcionales (RNF)

- RNF1 - Seguridad: Las contraseñas de los usuarios se almacenarán encriptadas utilizando algoritmos de hashing robustos (bcrypt).
- RNF2 - Usabilidad/Responsive: La interfaz se adaptará fluidamente a pantallas de escritorio, tablets y móviles.

- RNF3 - Portabilidad: La aplicación deberá poder desplegarse en cualquier sistema operativo que soporte Docker sin necesidad de instalar dependencias locales (Python/Node).
- RNF4 - Rendimiento: Las respuestas de la API no deberán exceder los 200ms para operaciones de lectura estándar.

9. Áreas del ciclo formativo desarrolladas

Este proyecto integra competencias de los siguientes módulos del ciclo DAW:

- 1. Desarrollo Web en Entorno Servidor (DWES):** Creación de la API con Python, gestión de base de datos PostgreSQL, autenticación y lógica de negocio.
- 2. Desarrollo Web en Entorno Cliente (DWECE):** Desarrollo de la SPA (Single Page Application) con React, gestión del estado, consumo de API asíncrona.
- 3. Diseño de Interfaces Web (DIW):** Maquetación responsive con Tailwind CSS, usabilidad y experiencia de usuario (UX).
- 4. Despliegue de Aplicaciones Web (DAW):** Configuración de Docker, Docker Compose, variables de entorno y puesta en producción.